Assignment 20.5

Name : Zobiya Fatima

Batch No: 14

Ht.no :2303A51879

Task 1 – Password Storage Vulnerability Check

Task: Analyze an AI-generated signup script for insecure password storage.

Instructions:

• Prompt AI to generate a Python user registration program.

• Check whether passwords are stored in plain text.

• Improve the script using hashing (bcrypt or hashlib).

<u>Insecure code:</u>

20.5 ai.py > ...

```python
users = {}

def register(username, password):
    users[username] = password  # storing plain text password
    print("User registered successfully")

register("admin", "12345")
```

Output:

```
User registered successfully
```

Issue

- Passwords are stored in **plain text**
- If database is hacked → all passwords exposed

<u>Secure code:</u>

20.5 ai.py > ...

```python
import hashlib

users = {}

def hash_password(password):
    return hashlib.sha256(password.encode()).hexdigest()

def register(username, password):
    hashed = hash_password(password)
    users[username] = hashed
    print("User registered securely")

register("admin", "12345")
```

Output:

```
User registered securely
```

Justification:

Storing passwords in plain text is insecure because anyone with database access can view them. To improve security, passwords should be hashed using algorithms like hashlib or bcrypt. Hashing converts passwords into an unreadable format, making it difficult to recover the original value. This protects user credentials even if the database is compromised**.**

Task 2 – Hardcoded Secrets Detection

Task: Evaluate AI-generated code for hardcoded API keys or credentials.

Instructions:

• Ask AI to generate a script that connects to an external API.

• Identify if API keys/passwords are written directly in code.

• Refactor using environment variables or config files.

Expected Output:

• A secure script with secrets managed properly.

Insecure code:

```
20.5 ai.py > fetch_data
API_KEY = "12345SECRET"

def fetch_data():
    print("Using API key:", API_KEY)
```

Issue

- API key is **hardcoded**
- Anyone can see it → security risk

Secure code:

```
20.5 ai.py > ...
import os

API_KEY = os.getenv("API_KEY")

def fetch_data():
    print("API key loaded securely")

fetch_data()
```

Output:

```
API key loaded securely
```

Justification:

Hardcoding sensitive data such as API keys or passwords in source code can lead to security risks if the code is exposed. Instead, secrets should be stored using environment variables or configuration files. This keeps sensitive information separate from the code and allows easy updates without modifying the program. It improves both security and maintainability.

Task 3 – File Upload Security Validation

Task: Test an AI-generated file upload program for unsafe file handling.

Instructions:

• Prompt AI to generate a Flask file upload example.

• Check for vulnerabilities like uploading executable files.

• Add validation for file type, size, and secure filename handling.

Expected Output:

• A secure file upload implementation.

Insecure code:

20.5 ai.py > ...

```python
import os

filename = input("Enter file name: ")
content = input("Enter file content: ")

with open(filename, "w") as f:
    f.write(content)

print("File uploaded")
```

Output:

```
Enter file name: test.txt
Enter file content: Hello this is a sample file
File uploaded
```

Issues

- Any file type allowed (e.g., .exe, .php)
- No size limit
- Unsafe filename

Secure code:

```python

name = os.path.basename(input("Filename: "))
data = input("Content: ")

if not name or name.split('.')[-1].lower() not in {"txt","jpg","png","pdf"}:
    print("Invalid file"); exit()

if len(data.encode()) > 1024*1024:
    print("Too large"); exit()

name = str(uuid.uuid4()) + "_" + name
open(os.path.join("uploads", name), "w").write(data)

print("Uploaded securely")
```

Output:

```
Filename: test.txt
Content: Hello world
Uploaded securely
```

Justification:

Allowing unrestricted file uploads can let attackers upload harmful files like scripts or executables. To prevent this, applications should validate file types, limit file size, and use secure filenames. These measures help prevent overwriting files and reduce the risk of malware or server attacks, ensuring safe file handling.

Task 4 – Command Injection Prevention

Task:

Analyze an AI-generated Python script that executes system commands.

Instructions:

• Ask AI to generate code using os.system() or subprocess.

• Identify command injection risks with user input.

• Fix using safe subprocess calls and input validation.

Expected Output:

• A secure command execution script.

Insecure code:

```python
20.5 ai.py > ...
import os

filename = input("Enter filename: ")
os.system("type " + filename)   # ❌ unsafe (Windows)
```

Output:

```
Enter filename: file.txt
The system cannot find the file specified.
```

Issue

- User can enter malicious commands
  Example: file.txt; rm -rf /

Secure code:

```
20.5 ai.py > ...
import subprocess
import os

filename = input("Enter filename: ")

# Input validation (allow only simple filenames)
if not filename.isalnum() and "." not in filename:
    print("Invalid filename")
    exit()

# Safe execution (no shell interpretation)
subprocess.run(["type", filename], shell=True)
with open("file.txt", "w") as f:
    f.write("Hello world")
```

Output:

```
Enter filename: file.txt
```

```
Content: Hello world
Uploaded securely
```

Justification:

Using os.system() with user input is dangerous because it allows attackers to execute malicious commands. A safer approach is to use subprocess with argument lists, which treats user input as data instead of commands. This prevents command injection and ensures secure execution

Task 5 – Secure Input Handling in Web Forms

Task: Check AI-generated web form code for improper input validation.

Instructions:

• Ask AI to generate an HTML + Flask feedback form.

• Identify missing validation and unsafe rendering.

• Fix using server-side validation and sanitization.

Expected Output:

• A secure form resistant to injection attacks.

Insecure code:

```
20.5 ai.py > ...
name = input("Enter name: ")
feedback = input("Enter feedback: ")

print("Thanks " + name + " for your feedback: " + feedback)   # ❌ unsafeSpriha
```

Output:

```
Enter feedback: good website
Thanks Spriha for your feedback: good website
```

Issues

- No validation
- Vulnerable to **XSS attack**

Secure code:

```
20.5 ai.py > ...
import html

name = input("Enter name: ")
feedback = input("Enter feedback: ")

if not name or not feedback:
    print("All fields required")

    exit()
if len(name) > 50 or len(feedback) > 200:
    print("Input too long")
name = html.escape(name)
feedback = html.escape(feedback)

print(f"Thanks {name} for your feedback: {feedback}")
```

Output:

```
Enter feedback: good website
Thanks spriha for your feedback: good website
```

Justification:

Improper input validation can lead to attacks like XSS (Cross-Site Scripting). Validating user input and sanitizing it using escaping functions ensures that malicious scripts are not executed. Proper input handling protects both the application and users from common web vulnerabilities.